LLM Ghostbusters: Surgical Package Hallucination Suppression via Adaptive Unlearning

Joseph Spracklen[†]
University of Texas at San Antonio

Pedram Aghazadeh[†]
University of California San Diego

Farinaz Koushanfar
University of California San Diego

Murtuza Jadliwala
University of Texas at San Antonio

arXiv:2605.01047v2 [cs.CR] 21 Sep 2026

Abstract

Hallucinations remain an unsolved problem for LLMs, and *package hallucinations* are a particularly dangerous instance of this phenomenon. Package hallucinations occur during code generation when a model fabricates non-existent software packages, recommending imports and installation commands for fictional libraries. This creates a critical supply-chain vulnerability; an attacker can proactively register such packages on public registries with malicious payloads that are subsequently installed and executed by developers or autonomous agents. These hallucinations enable a class of package confusion attack known as *slopsquatting*. To address this issue, we present **Adaptive Unlearning** (AU), a post-deployment framework that surgically suppresses package hallucinations while preserving general model utility. AU introduces a hybrid token-level objective that simultaneously reinforces valid outputs and suppresses hallucinated ones. Combined with an adaptive discovery loop that continuously surfaces new hallucination-inducing contexts without human supervision, AU enables generalization to unseen prompts and hallucinations. We demonstrate that AU reduces package hallucination rates by **88%**, while maintaining performance on standard coding benchmarks. Our analysis shows that distributional changes are concentrated on package-related generations, leaving general coding behavior largely unaffected and confirming that AU's effect is isolated to the targeted distribution. AU relies entirely on model-generated data and requires no human annotation, representing a post-deployment hallucination mitigation framework.

1 Introduction

Large language models (LLMs) have moved from research artifacts into production infrastructure, with billions of daily interactions now routed through systems that synthesize information, answer questions, and generate code on behalf of users and downstream applications. This trajectory has made LLMs a load-bearing component of software supply chains, developer workflows, and enterprise decision systems and, in turn, has made their failure modes security-relevant. Most notable among these failures are *hallucinations*, fluent and plausible outputs that are nonetheless factually incorrect, unsupported by the input, or fabricated.

In some cases, hallucinations have been shown to be exploitable vulnerabilities rather than merely quality defects. A prime example is a use case specific to code generation called package hallucination. When an LLM fabricates a package name that does not exist in the respective package repository, the hallucinated identifier becomes an attack target: an adversary can register the fabricated name with a malicious payload and wait for downstream users or autonomous coding agents to download and install it. This is the basis of *slopsquatting* and *package confusion* attacks against LLM-generated code [50]. Any model output that names an external resource (packages, URLs, API endpoints, registry identifiers) creates an analogous surface whenever the hallucination is predictable or reproducible. In this setting, hallucination rate is a measure of attack-surface volume.

Will this surface shrink to zero as models improve? The evidence says no. Scaling model size and data, long the dominant recipe for capability gains [19, 26], is running into both practical limits [53], and it does not appear to eliminate hallucinations [63]. TruthfulQA [32] documents cases where larger models become *less* truthful, and recent theoretical work argues that hallucinations are not implementation bugs but artifacts of training a model to maximize likelihood rather than acknowledge uncertainty [25]. Post-training refinements, such as RLHF [44], RLAIF [3], GRPO [48], are likewise insufficient. Recent analyses indicate that RL-based post-training predominantly reshapes output preferences and sampling behavior rather than altering the pretrained knowledge representations that generate hallucinations in the first place [58, 60].

Package hallucinations have emerged as a distinct and active research field and serve as an unofficial benchmark for hallucination progress due to their clear relevance as an active security threat and ease of detection. Since the first large scale

[*]Equal contribution.

characterization of the phenomenon [50], a growing body of dedicated work has extended the measurements to new metrics [27], additional ecosystems [64], and to additional prompt variations [52]. A parallel line has produced countermeasures spanning adversarial fuzzing [62], decoding-time constraints [35], localized model editing [34], and calibrated detection [18]. Crucially, this problem has not been overtaken by model scaling; recent research on the latest frontier models reports hallucination rates have not improved since the original study [9] and identifies a large common set of hallucinated package names that appear in every evaluated frontier model. These latest results confirm that package hallucinations are a persistent and exploitable phenomenon that demands intervention beyond waiting for models to improve organically.

Comprehensive surveys [22, 61] and measurement studies [30, 41] confirm the pattern empirically: hallucinations persist across tasks, domains, and model generations, at rates high enough that frontier vendors now treat hallucination reduction as a primary release-gating objective. OpenAI's GPT-5 announcement centers a 45–80% improvement on factuality benchmarks [43]; Anthropic's Claude 4 release [1] and Google's Gemini 3 release [15] each foreground hallucination behavior as a primary deployment concern, with Gemini 3 exhibiting an 88% hallucination rate on a subset of queries where the model answers confidently rather than deferring.

Taken together, these observations motivate a different class of intervention. If hallucinations cannot be fully eliminated during pre-training and are only partially masked by post-training alignment, then reducing their security impact requires *post-deployment refinement*. Mechanisms that precisely suppress specific unwanted behaviors in an already-trained model, without retraining from scratch while avoiding collateral damage to capability. This is the problem we address in this work.

1.1 The Post-Deployment Refinement Gap

An ideal post-deployment refinement mechanism would satisfy four properties simultaneously:

1. **Precision.** Remove the targeted hallucination behavior with negligible degradation on unrelated capabilities.

2. **Generalization.** Suppress hallucinations across prompt variations, rather than memorizing fixes for specific prompt–output pairs that an adversary or user could trivially rephrase.

3. **Data efficiency.** Operate on synthetic data generated by the model itself, without requiring human-annotated hallucination corpora.

4. **Scalability.** Support continuous refinement as new hallucinations are reported in production, on timescales compatible with release cycles

Existing approaches fall short of these requirements. Privacy-focused machine unlearning methods [5, 6, 16] target removal of specific training examples to satisfy regulations like GDPR, but hallucinations are not discrete facts to be forgotten, they are rather emergent behaviors distributed throughout the model. Knowledge editing techniques like ROME [39] and MEMIT [40] can update specific factual associations, but they operate on the premise of *replacing* rather than *suppressing unwanted generation patterns*. Retrieval-augmented generation [29] and inference-time verification methods [38] add computational overhead and architectural complexity while failing to address the root cause in the model's learned representations. Constitutional AI and self-consistency approaches rely on a model's ability to critique itself, a capability that fails when models hallucinate confidently [21, 59].

Recent work on Partial Model Collapse (PMC) [46] offers a promising avenue in Machine Unlearning. PMC recognizes that the same collapse dynamics that threaten recursive training regimes can be deliberately induced in a *controlled* fashion to remove targeted content while preserving overall utility, turning an uncontrolled failure mode into a precise unlearning mechanism. However, PMC is designed for traditional machine-unlearning scenarios in which the target set is finite, well-defined, and known in advance (e.g., a specific set of training documents to forget due to privacy concerns). Hallucination reduction violates all three assumptions: the space of hallucination-inducing prompts is open-ended, the set of hallucinated outputs a model will produce is unknown until elicited, and new failure modes surface continuously as deployment contexts shift. Closing the gap to a deployable post-deployment refinement mechanism therefore requires two additional components PMC does not provide: (i) a mechanism to *discover* hallucination-inducing prompts and (ii) a training scheme that generalizes suppression to prompt variations the method has not yet seen.

1.2 Our Approach.

We address this gap with **Adaptive Unlearning (AU)**[1], a closed-loop post-deployment refinement framework that directly supplies the two ingredients other methods such as PMC lack. First, an adaptive prompt-mutation loop that continuously elicits new hallucination-inducing contexts from the model itself, rather than assuming a fixed target set. Second, a token-level hybrid objective that reinforces valid outputs while suppressing hallucinated ones within the same sequence, generalizing suppression to prompt variations the method has not seen. AU extends the Model Collapse and NPO frameworks [46, 60] to the open-ended prompt space inherent in hallucination reduction, operates entirely on synthetic self-generated data, and requires no human annotated corpora. In our experiments, AU reduces hallucination rates

[1] Anonymized code available at : https://anonymous.4open.science/r/Adaptive-Unlearning-952E

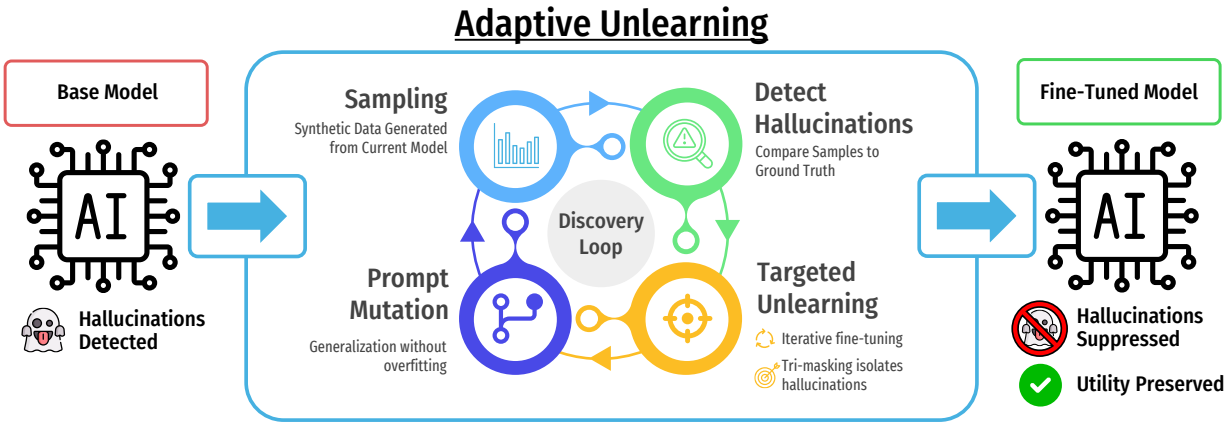


Figure 1: **Adaptive Unlearning pipeline.** The four-stage discovery loop, Sampling, Detection, Targeted Unlearning, and Prompt Mutation, iteratively suppresses hallucinated package tokens while preserving general coding utility. The full method is detailed in §3.

by over **88%** on unseen prompt variations while preserving utility on established code-generation benchmarks, as detailed in §3 and as shown in Fig. 1.

1.3 Contributions

Our work makes the following contributions:

1. **Adaptive Unlearning:** We introduce AU, a novel closed-loop post-deployment framework that surgically suppresses package hallucinations without full retraining, not requiring human annotation, and without a pre-specified forget set. AU is the first unlearning method to couple an adaptive hallucination-discovery loop with a hybrid token-level objective, enabling suppression that generalizes to unseen prompts rather than memorizing fixes for known ones. In package hallucination tests, a setting with direct security consequences via slopsquatting, AU reduces hallucination rates by 88% while preserving coding benchmark performance, outperforming all evaluated baselines on the joint hallucination-utility tradeoff.

2. **Tri-mask token routing:** AU uses a novel three-valued gradient-routing mechanism that partitions the tokens into three sets: valid-content tokens, hallucinated tokens, and context tokens. We use distinct loss terms for each. Unlike the binary masking used in prior unlearning work [11, 36], tri-masking enables surgical edits within a single sequence without affecting adjacent tokens. This fine-grained routing is what allows the CE and NPO objectives run on separate token populations without interfering.

3. **Adaptive Prompt Mutation:** We introduce a closed-loop mutation strategy that continuously elicits hallucination-inducing contexts from the model itself, retiring exhausted prompts in favor of semantically related variants. This extends existing unlearning methods from finite, pre-specified forget sets to the open-ended prompt space inherent in real-world settings, enabling suppression of hallucination patterns rather than memorization of prompt-output pairs.

4. **Nested training for stable unlearning.** We propose a nested-loop training structure with outer resampling epochs and inner unlearning epochs that caches synthetic samples for multiple inner steps before regeneration. This directly addresses the instability of prior approaches that resample every step and therefore never consolidate knowledge. To the best of our knowledge, this nested cache-refresh scheme has not previously been applied to collapse-based unlearning, and our ablations show it is crucial for stable hallucination reduction.

5. **Systematic joint evaluation.** We provide, to our knowledge, the first empirical comparison of unlearning methods for any type of hallucination reduction, evaluated jointly across hallucination rate, distributional drift, and coding utility. This joint view is necessary: methods that appear effective on any single axis often fail on another, and the tradeoff is only visible when all three are reported together.

2 Background

2.1 Hallucinations in LLMs

Hallucinations denote outputs that are linguistically fluent and superficially plausible but factually incorrect, unsupported by the input, or entirely fabricated. Their causes are well-studied [24, 32, 54]: training corpora contain noisy and contra-

dictory claims that models absorb and reproduce confidently, the next-token prediction objective rewards fluency more directly than truthfulness, and exposure bias at decoding time compounds small errors as the model conditions on its prior mistakes. These factors make hallucinations persistent across tasks, domains, and model generations.

2.2 Hallucination Reduction

Existing mitigation approaches divide into **proactive** methods that modify the model and **reactive** methods that intervene at inference. Proactively, data curation and de-duplication during pretraining [28, 37] and RLHF [44] dominante. RLHF improves truthfulness on benchmarks like TruthfulQA but requires expensive annotation and struggles with rare failure modes; Constitutional AI [3] cuts annotation cost via self-critique, but presupposes the model can detect its own errors, a capability that fails precisely when it hallucinates confidently.

Reactively, RAG [29] grounds responses in external knowledge bases at the cost of latency, architectural complexity, and the burden of maintaining the external corpus. DoLa [8] contrasts logits across transformer layers for 12-17% gains on TruthfulQA without fine-tuning. Chain-of-thought [56] and self-consistency [55] improve reasoning but not factuality directly, and incur multiple forward passes per query. Constrained decoding enforces structural correctness only and is limited to narrow output formats. None of these enable *post-deployment, targeted* removal of specific hallucination types: proactive methods require full retraining and alter global behavior, while reactive methods treat symptoms without touching the weights that produce them. *Adaptive Unlearning* fills this gap.

2.3 Unlearning in LLMs

As LLMs are often trained on massive corpora, they may inadvertently memorize sensitive, private, or outdated information. *Machine unlearning* removes or suppresses the influence of specific training data or knowledge in a trained model without retraining from scratch [42], with applications to privacy compliance and post-hoc model correction. *Exact unlearning* produces a model that behaves as if the target data had never been seen, but requires full retraining and is intractable at scale [42]. Recent work therefore focuses on empirical algorithms that fine-tune the model to forget targeted content efficiently [23].

One approach is *gradient ascent*: fine-tuning the model by *maximizing* loss on the data to forget, reducing its confidence in that completion. This has been successfully applied to LLMs [23], but suffers from *catastrophic collapse*. As the model is pushed away from certain knowledge, the overall performance of the model degrades. An overshooting optimizer can erase neighboring capabilities along with the target knowledge, often resulting in gibberish outputs [60].

Negative Preference Optimization (NPO) [60], inspired by DPO [45], addresses this instability by structuring the objective as a preference over the target data rather than as raw error maximization, scaling unlearning to realistic dataset sizes without collapse. While the original NPO method used a frozen base model as a reference in the loss function, this causes the fine-tuned model to be compared to a stale base model after many epochs, a problem which reference-free variants have addressed [13]. Conservative alternatives such as logit suppression [20] instead localize updates to preserve surface-level performance, but often achieve only partial unlearning. The model refrains from verbatim reproduction while still "knowing" the information, leaking it through paraphrase that can itself manifest as new hallucinations.

A separate line of work repurposes a known failure mode. Iteratively training a generative model on its own outputs induces *model collapse*, a progressive loss of distributional diversity in which the tails of the output distribution are forgotten and the model degenerates into repetitive text [4, 12, 49]. The effect is irreversible. Further training on synthetic data cannot recover the lost diversity [14, 47]. *Partial Model Collapse* (PMC) [46] triggers this process in a targeted way. It removes specific knowledge rather than degrading the model as a whole, and forms the methodological starting point for our approach in §3.

2.4 Hallucination Reduction via Unlearning

Recent work has begun to treat hallucinations as an unlearning target, removing the incorrect associations that produce it. The goal here is not privacy or regulatory compliance, but factual accuracy through forgetting incorrect knowledge or disabling spurious generation pathways.

Tan et al. [51] characterize the central dilemma as a trade-off between aggressive unlearning (which harms utility) and conservative unlearning (which leaves the model fluently filling the gap with new fabrications). They integrate refusal behavior into the unlearning objective, so the model responds with a safe fallback (e.g. "I'm not sure") rather than a new incorrect statement.

Despite extensive work on hallucination mitigation, only a small and recent subset of works casts hallucination suppression as an unlearning problem. Notably, most approaches in this direction have focused on multi-modal LLMs, leaving text-only hallucination via unlearning comparatively unexplored [31, 57].

3 Method

Adaptive Unlearning suppresses package hallucination through a closed-loop system. The loop iteratively discovers hallucination-inducing prompts, samples the model's outputs, and applies token-level unlearning to hallucinated completions. It extends PMC from a finite, enumerable forget set

to the open-ended prompt space that hallucination reduction requires. Rather than removing discrete facts, AU repeatedly trains the model on its own hallucinated outputs to induce a controlled distributional collapse at hallucination-inducing contexts, while adaptive prompt mutation continuously supplies new instances so the model suppresses the pattern rather than memorizing prompt-output pairs.

3.1 Problem Formulation

Let $\mathcal{M}_\theta$ denote a language model with parameters θ, and let $\mathcal{P}$ be a distribution over prompts that induce hallucinated packages.

For $p \sim \mathcal{P}$, the model emits a token sequence $y = (y_1, \ldots, y_T)$. A detection oracle O labels each package reference in y as valid or hallucinated, yielding position sets $\mathcal{T}_{\text{forget}}$ and $\mathcal{T}_{\text{retain}}$, with all remaining positions forming $\mathcal{T}_{\text{reg}}$. We seek θ that suppresses generation at $\mathcal{T}_{\text{forget}}$ under prompt variation, preserves utility on general coding tasks, and generalizes to prompts unseen during training.

Standard unlearning assumes a forget set that is finite and known in advance. Here neither $\mathcal{P}$ nor the hallucinated spans it elicits can be enumerated ahead of time, and both shift as θ changes. Both must be discovered during training, which is the requirement that motivates the closed loop in §3.5 and §3.6.

3.2 Loss Functions

AU's objective combines three terms. Cross-entropy (CE) reinforces tokens the oracle validates. NPO suppresses tokens it rejects, and a scaled cross-entropy term anchors the remaining tokens to the base distribution. The tri-mask in §3.4 ensures all three terms act on disjoint token subsets. AU extends PMC [46] from finite to open-ended forget sets, and we summarize PMC's formal objective in Appendix E.

The cross-entropy loss function applies standard next-token prediction:

$$\mathcal{L}_{\text{CE}}(\theta) = -\sum_{\{t : m_t = 1\}} \log p_\theta(y_t \mid y_{<t}, x) \tag{1}$$

where x is the input prompt and $y_{<t}$ is the preceding token sequence. This objective reinforces valid package tokens and structural code elements, serving as the primary utility-preserving component of the hybrid loss.

NPO [60] frames unlearning as preference optimization, providing more stable gradients than gradient ascent. We apply it at token granularity,

$$\mathcal{L}_{\text{NPO}}(\theta) = -\mathbb{E}_{y^-}\left[\log \sigma\left(-\beta \log \frac{\pi_\theta(y^- \mid p)}{\pi_{\text{ref}}(y^- \mid p)}\right)\right] \tag{2}$$

where y^- are hallucinated completions, π_{ref} is the frozen base model, β is a temperature controlling suppression strength, and σ is the sigmoid function.

3.3 Tri-Masking

Tri-masking routes gradients at the token level, determining which positions are reinforced, suppressed, or held near the base distribution. Prior masking approaches in unlearning are binary. Tri-masking adds a third category, and the resulting mask $\mathbf{m} = (m_1, \ldots, m_T) \in \{0,1,2\}^T$ assigns one value to every token position.

- **Mask = 0 (Regularize):** Tokens receive a regularization weight in the form of cross-entropy loss scaled by λ_{reg}. This is applied to all tokens that are not package names, including prompts, code, and punctuation. This has the effect of anchoring a model's distribution at non-target positions, preventing parameter drift that otherwise causes output degeneration.

- **Mask = 1 (Reinforce):** Tokens receive standard reinforcement through cross-entropy loss. Applied to correct tokens, which in our use case are the valid package names.

- **Mask = 2 (Suppress):** This applies to hallucinated tokens which receive suppression gradients via NPO loss. In our use case this represents hallucinated package names.

The tri-mask $\mathbf{m} = (m_0, m_1, m_2)$ is generated dynamically for each training sample based on the domain-specific hallucination detection. For package hallucinations, our detection pipeline identifies non-existent package names and marks those tokens with mask = 2, marks valid packages with mask = 1, and marks all other tokens with mask = 0. It is important to note that this tri-masking process is only necessary during the fine-tuning of the model and is not required during regular model use.

A critical finding on contextual and punctuation tokens was discovered as our initial implementation assigned no gradient to any non-package token. This led to a severe degradation in the model's coding ability. We then implemented a regularization term that anchored these critical tokens to the base model, preventing incoherent responses and repetitive over-generation, where the model continued producing tokens indefinitely.

3.4 The Adaptive Unlearning Objective

The full Adaptive Unlearning objective combines the cross-entropy and NPO losses introduced above through the tri-mask, applying each to a disjoint token population within the same sequence. Concretely, for a tokenized sequence $y = (y_1, \ldots, y_T)$ with tri-mask $\mathbf{m} = (m_1, \ldots, m_T) \in \{0,1,2\}^T$,

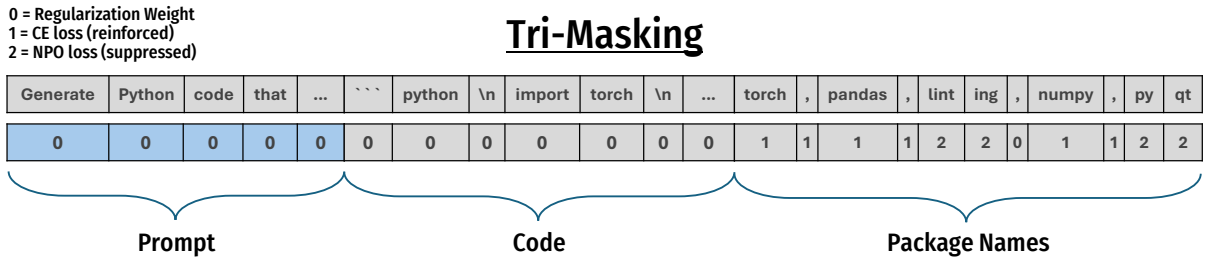


Figure 2: Token-level tri-masking applies targeted loss functions across every sample for precise suppression and reinforcement.

define the partitioned token-position sets

$$\mathcal{T}_{\text{reg}} = \{t : m_t = 0\}, \tag{3}$$

$$\mathcal{T}_{\text{retain}} = \{t : m_t = 1\}, \tag{4}$$

$$\mathcal{T}_{\text{forget}} = \{t : m_t = 2\}. \tag{5}$$

The Adaptive Unlearning objective is then:

$$\mathcal{L}_{\text{AU}}(\theta) = \lambda_{\text{retain}} \mathcal{L}_{\text{CE}}^{\mathcal{T}_{\text{retain}}}(\theta) + \lambda_{\text{forget}} \mathcal{L}_{\text{NPO}}^{\mathcal{T}_{\text{forget}}}(\theta) + \lambda_{\text{reg}} \mathcal{L}_{\text{CE}}^{\mathcal{T}_{\text{reg}}}(\theta), \tag{6}$$

where each term is the corresponding loss from §3.2 restricted to the indicated token positions.

The retain term $\mathcal{L}_{\text{CE}}^{\mathcal{T}_{\text{retain}}}$ inherits PMC's self-training mechanic. Rather than reinforcing externally curated ground-truth tokens, it reinforces the valid package tokens the current model π_θ already emits and that O confirms. Iteratively training on these self-generated, oracle-validated tokens drives the conditional distribution on package-recommendation prompts toward the outputs the model emits *and* that survive verification.

This is a per-token specialization of PMC's curated-self-training (7), with curation operator $\mathcal{C}$ instantiated by the oracle O filter rather than a global preference function. The forget term $\mathcal{L}_{\text{NPO}}^{\mathcal{T}_{\text{forget}}}$ pushes probability mass off the complementary subset, meaning tokens the model produces but the oracle rejects. The regularization term $\mathcal{L}_{\text{CE}}^{\mathcal{T}_{\text{reg}}}$ anchors the model on the remaining structural and contextual tokens near the base model.

A partition contributes only when non-empty. A completion containing only hallucinated packages leaves $\mathcal{T}_{\text{retain}} = \emptyset$ and drops that term for the sample, and the same holds for the other two. One objective therefore covers forget-only, retain-only, and mixed sequences without special-casing.

Interaction with the tri-mask. Disjointness is what makes (6) well-posed. Because the three terms act on non-overlapping token positions, the gradients from $\mathcal{L}_{\text{CE}}^{\mathcal{T}_{\text{retain}}}$ (pushing probability mass toward valid package tokens) and $\mathcal{L}_{\text{NPO}}^{\mathcal{T}_{\text{forget}}}$ (pushing probability mass away from hallucinated package tokens) cannot conflict at a single token. Applying CE and NPO sequence-wide instead produces opposing gradients on the same logit vector at the same step, the mechanism behind the catastrophic-collapse instability documented for naïve gradient ascent [60].

Disjointness is an assumption. It holds for package identifiers, which sit in delimited import positions and rarely influence the surrounding code. It weakens when a hallucinated name collides with a legitimate identifier, or shares subword pieces with a valid name so that suppression leaks. It fails entirely for targets that spread incorrectness across a clause rather than a delimited span, which we treat as out of scope.

3.5 Nested Training Loops

PMC-based unlearning cannot run on a fixed dataset. Inducing collapse requires regenerating model outputs as θ changes. However, regenerating samples every epoch is unstable, because the model does not register a given sample often enough to consolidate which patterns to suppress. AU therefore separates resampling from optimization across two timescales.

Outer Epochs (N_{outer}) control resampling frequency. At the start of each, the current model generates fresh completions for all active prompts in inference mode, and the detector labels them to build tri-masks. This tracks the distribution as it shifts under training.

Inner Epochs (N_{inner}) enable knowledge consolidation. The model trains repeatedly on the cached completions from the current outer epoch, letting gradients accumulate and reinforce the suppression pattern before the samples are regenerated. This prevents the usual instability where the model constantly "chases" shifting samples.

This design choice is critical for hallucination reduction. In ablation studies, training without inner epochs (equivalent to $N_{inner} = 1$) failed to reduce hallucination rates. Hallucination rate fell monotonically at $N_{\text{inner}} \geq 10$.

3.6 Prompt Mutation Strategy

Suppression must generalize beyond the prompts it was trained on. A fixed prompt set risks teaching the model to avoid hallucinating on those exact inputs while leaving the underlying behavior intact.

AU mutates prompts as they stop producing signal. A prompt is *exhausted* when N sampled completions contain no hallucinated package, or when it has been trained for five outer epochs, whichever comes first. The model is then shown the exhausted prompt and instructed to write a reworded version of it $p' = \textsc{Mutate}(p)$, always from the original seed rather than the latest variant so that mutations do not compound. The rewrite is constrained to preserve the task's library requirements and technical goal while varying the phrasing. The variant should elicit different code, but code requiring the same packages. Mutation resets the exhaustion state and clears the cached code. This forces the model to generalize prior training iterations to name valid packages and suppress hallucinated ones rather than just memorizing the original wording.

After six rounds of mutation, hallucination rate on held-out prompts was 3.9%, against 12.7% for training on the original prompts alone.

3.7 Training

The complete AU training procedure integrates all components, as seen in Fig. 1. This procedure ensures that: (1) the model encounters diverse hallucination-inducing contexts through mutation, (2) training is stable through nested epochs, (3) gradient flow is precise through tri-masking, and (4) both suppression and preservation occur through the mixed loss objective.

4 Experimental Evaluation

We evaluate Adaptive Unlearning on package hallucination in code generation, which pairs verifiable ground truth with a live attack surface.

4.1 Domain and Detection

Package hallucinations arise when a code-generating model emits references to libraries absent in the target ecosystem's registry [50]. A model asked for a Python plotting script may import the legitimate `matplotlib` alongside a fabricated `quickplot`. This failure mode is particularly dangerous for four reasons:

- **Syntactic validity.** Hallucinated imports are syntactically well-formed and indistinguishable from legitimate ones at the parser level.
- **Semantic plausibility.** Fabricated names frequently follow natural naming conventions (e.g., `fastjson`, `easyplot`) and appear credible to human reviewers.
- **Attack surface.** Hallucinated package names enable *package confusion* and *slopsquatting* attacks [50]. an adversary who observes (or predicts) hallucinated names can publish malicious packages under those names, which are then available to be installed and executed by downstream users of the model.
- **Misplaced trust.** As developers increasingly adopt model-generated code with limited review, hallucinated dependencies provide a low risk/high reward injection vector into otherwise legitimate software supply chains.

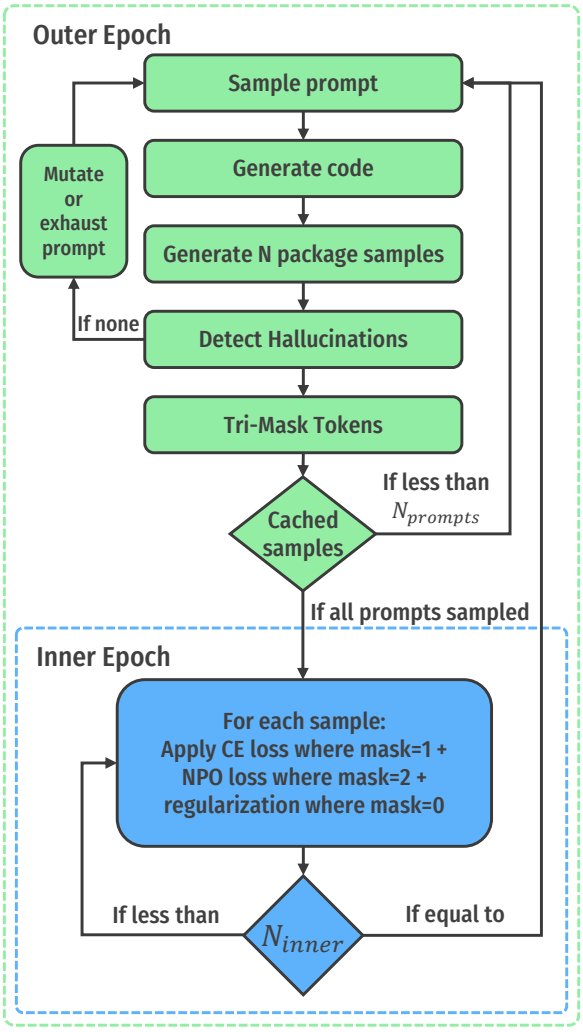


Figure 3: **Nested outer/inner epoch structure.** Nested outer/inner epoch structure. Outer epochs regenerate samples and invoke prompt mutation; inner epochs train on cached samples via the tri-masked hybrid objective, stabilizing the unlearning loop. This process is detailed in §3.5.

The domain also has properties that are rare for unlearning evaluation. Package existence is deterministic, since a name either resolves against the PyPI index or does not. The prompt space is open-ended, as arbitrarily many specifications elicit package references. And the failure mode is operationally live

rather than hypothetical.

We detect hallucinations by cross-referencing outputs against a canonical PyPI snapshot. Given a prompt p, the model produces completion y. We parse y for explicit `pip install` directives, then separately query the model for the packages required to run y and parse the response r for identifiers. Each identifier is resolved against the snapshot, and we emit the token-level mask $\mathbf{m}$ over r as described in §3.3. A visual description of the pipeline is in Appendix G.

4.2 Setup

Models. We evaluate two instruction-tuned models from the DeepSeek-Coder family [17], `DeepSeek-Coder-7b-instruct-v1.5` (dense, 7B) and `DeepSeek-Coder-V2-Lite-Instruct` (MoE, 16B total and roughly 2.4B active per token). The pair spans dense and sparsely activated architectures at a scale comparable to prior unlearning work [46, 60]. All runs use full-parameter fine-tuning rather than low-rank adaptation, so results reflect the objective rather than adapter capacity. Experiments ran on AMD MI210 and MI300 GPUs.

Dataset. We curate a set of 20 code-generation prompts drawn from the benchmark introduced by Spracklen et al. [50]. Prompts are selected subject to two criteria: (i) under the base model's initial sampling, the completion contains at least one valid and one hallucinated package, ensuring the prompt is a meaningful stressor; and (ii) the set collectively covers a diverse range of programming tasks and application domains, mitigating topical bias. We intentionally selected prompts that elicit a large number of hallucinations to thoroughly test our method, which explains why our baseline results being much larger than in the original study. For testing, we used ChatGPT to develop 2 variants of each initial coding prompt. This 40 prompt test set is completely held-out of training and enables generalized testing on unseen prompts. The full list of initial coding prompts can be found in Appendices C and F.

Configuration. Batch size 8, maximum sequence length 1024, six mutations per prompt, five cached samples per outer epoch, and exhaustion at five outer epochs or zero hallucinations. Loss weights are $(\lambda_{\text{retain}}, \lambda_{\text{forget}}, \lambda_{\text{reg}}) = (1.00, 1.25, 1.00)$, selected over a small grid reported in Appendix D. All methods share the same generation, parsing, and detection pipeline, and differ only in the loss applied and whether samples are regenerated online.

4.3 Baselines and Ablations

We organize our experimental comparisons along two axes. The **baselines** situate Adaptive Unlearning against the prior art in hallucination suppression and unlearning. The **ablated variants** isolate the contribution of individual components of our method by zeroing them out.

Methods.

1. **Base.** The unmodified DeepSeek-Coder model, establishing the reference hallucination rate and unmodified coding performance.
2. **Gradient Ascent (GA).** Direct gradient ascent on the negative log-likelihood of hallucinated tokens, the canonical first-line unlearning baseline.
3. **Negative Preference Optimization (NPO).** The preference-based unlearning objective of Zhang et al. [60], applied to suppress hallucinated tokens.
4. **PMC.** The self-training distribution-collapse method of Scholten et al. [46], which fine-tunes on the model's own generations selected against a preference function.
5. **Adaptive Unlearning (ours).** The full method: a hybrid token-level objective routed through a tri-mask, embedded in an adaptive prompt-mutation loop with nested-epoch sample caching.

Ablated variants. To attribute AU's performance to its individual components, we evaluate two ablations that zero out one loss term while holding the rest of the method fixed:

- **AU CE-Only** ($\lambda_{\text{retain}} = 1.0$, $\lambda_{\text{forget}} = 0$): the NPO suppression term is removed; the model receives reinforcement gradients on valid package tokens but no suppression on hallucinated ones.
- **AU NPO-Only** ($\lambda_{\text{forget}} = 1.0$, $\lambda_{\text{retain}} = 0$): the CE reinforcement term is removed; the model receives suppression gradients on hallucinated tokens but no reinforcement of valid ones.

We additionally examine the sensitivity of AU to two key hyperparameters of the adaptive scaffold: the number of inner epochs over cached samples, and the maximum number of prompt mutations per prompt. We return to all comparisons in §6.

4.4 Metrics

Following Spracklen et al. [50], each prompt receives two follow-up queries, one asking which packages are required to run the code and one asking which would be useful for the task. The first targets strict runtime dependencies and the second elicits optional suggestions.

Mode 1 (required). "Which packages are required to run this code?" Targeting strict runtime dependencies.

Mode 2 (helpful). "What packages would be useful to solve this task?" Eliciting optional and exploratory suggestions.

Our primary metric is hallucination rate, $\text{HR} = N_{\text{halluc}}/N_{\text{total}}$, over all identifiers emitted across both modes.

We measure utility on EvalPlus [33], whose expanded HumanEval+ and MBPP+ suites [2, 7] catch solutions that pass the original tests but fail on edge cases. We report pass@1 against the unmodified base model. To bound collateral damage, we also report KL-divergence from the base model on three response types, namely code samples, instruction following, and package recommendations. An ideal method isolates its updates to the package context.

5 Results

A useful post-deployment package hallucination mitigation method must do three things at once. It should reduce the hallucination rate, leave coding ability intact, and confine distributional change to the package generation rather than perturbing model broadly. A method that suppresses hallucinations at the cost of code quality has relocated the failure, not fixed it. Table 1 and Figure 4 therefore report all three axes together for every method.

We show that AU beats prior baselines on this tradeoff, quantify its cost in utility and drift, and attribute the effect to individual components (§6).

5.1 Primary Findings

Table 1 reports the cross-method comparison. **Adaptive Unlearning achieves the strongest overall tradeoff among the evaluated baselines with the lowest package hallucination rates on both models**. AU is the only method that simultaneously drives hallucination rates down to low single digits, preserves strong coding performance, and confines its largest behavioral changes to the package-generation setting rather than ordinary code generation. This is the correct lens for evaluating a post-deployment mitigation method. Reducing hallucinations alone is not enough if the model's coding ability collapses, and preserving utility alone is not enough if the model continues to emit exploitable fabricated package names.

AU reaches the lowest hallucination rate on both models, 2.56% on DeepSeek-7B and 6.13% on DeepSeek-16B, from base rates of 21.23% and 27.75%. On 7B this is an 88% relative reduction, and a 64% reduction against GA, the strongest non-AU baseline at 7.2%. All testing is executed on a held-out test set of prompts that were unseen during training.

Package hallucination suppression does not come from broad capability loss. On 7B model, AU attains the highest coding average in the table at 68.5%, 1.5% higher than the GA and NPO baselines, improving on three of four benchmarks with a small drop on MBPP+.PMC shows the alternative outcome. It cuts hallucination rate to 11.84% while its coding average falls from 66.98% to 51.65%, which is the failure mode the joint view exists to expose. The model trained via AU has clearly retained its coding proficiency while delivering state-of-the-art hallucination reduction performance.

AU's results on DeepSeek-16B, **while still the strongest among all evaluated methods**, show a wider gap in utility suppression than on DeepSeek-7B. Package hallucination rate drops to 6.1% (compared to 2.5% on 7B) at the cost of a 4.2% decline in average coding benchmark score (where 7B actually improved). We attribute the difference to MoE routing. With roughly 2.4B of 16B parameters active per token, each expert receives a sparse and fragmented unlearning signal, which makes consolidation harder and raises the risk of collateral updates to experts handling unrelated capabilities. A dense model updates every parameter on every forward pass. We do not test this mechanism directly and offer it as a hypothesis.

Distribution drift concentrates where AU targets it. On 7B model, AU induces 0.0796 KL on code against 1.4211 on package queries, **a ratio near 18 to 1**, with 0.9905 on instruction following.. AU is not the most conservative update in absolute terms where GA and PMC both perturb code less. However, the relevant property in this context is the *ratio*. The model is aggressive in pursuing inconsistencies in package inducing settings, providing strong updates to that specific part of the distribution while leaving the coding distribution relatively untouched. This is exactly the form of targeted change a practical post-deployment mitigation should aim for

Figure 4 complements Table 1 by showing that AU reaches this endpoint through a **stable trajectory** rather than an erratic or destructive one. In the left panel, package hallucination rate drops sharply early in training and then flattens, while HumanEval and MBPP remain essentially stable throughout. The right panel shows the same asymmetry distributionally. KL divergence stays near baseline for ordinary code generation and rises mainly in the package-related distribution.

Taken together, Table 1 and Figure 4 establish the main claims of the paper. AU is neither the most aggressive hallucination suppressor nor the most conservative fine-tuning procedure. It is the one that balances all three requirements, cutting package hallucination rate substantially while preserving coding utility and confining the update to package generation.

6 Ablation Studies

Having established the overall tradeoff in Table 1, we attribute it to individual components. AU differs from the prior baselines in two ways. It uses a hybrid token-level objective that both reinforces valid package names and suppresses hallucinated ones, and it embeds that objective in an adaptive scaffold of tri-masking, nested epochs, and prompt mutation.

	Hallucination Rate (%)		$D_{KL}(\pi_\theta \| \pi_{baseline})$			Coding Benchmarks (pass@1) ↑				
Variant	**Total ↓**	**Abs. Reduction ↑**	**Code↓**	**Instr.↓**	**Pkg. Avg.↑**	**HE**	**HE+**	**MBPP**	**MBPP+**	**Avg.**
DeepSeek-7B										
Base	21.23	0	–	–	–	66.5	62.22	74.1	**65.1**	66.98
GA	7.2	14.03	**0.0031**	**0.0679**	0.1529	66.5	61.6	**74.6**	**65.1**	66.95
NPO	14.17	7.06	**0.0191**	0.3565	0.7794	65.9	59.8	74.2	64.3	66.07
PMC	11.84	9.39	0.0060	0.1820	0.3622	59.8	56.1	48.9	41.8	51.65
AU (Ours)	**2.56**	**18.67**	0.0796	0.9905	**1.4211**	**73.2**	**64.6**	74.3	62.2	**68.57**
DeepSeek-16B										
Base	27.75	0	–	–	–	**81.1**	**75**	83.1	70.4	**77.4**
GA	13.75	14	0.0301	0.3236	**1.6029**	79.9	**75**	**83.9**	**70.6**	77.35
NPO	17.58	10.17	**0.0298**	**0.2691**	1.247	78	73.8	83.3	69.9	76.17
PMC	16.66	11.09	0.0521	0.2997	0.8683	72.6	66.5	71.4	60.8	67.82
AU (Ours)	**6.13**	**21.62**	0.0368	0.299	0.6782	73.2	69.5	81.2	68.8	73.17

Table 1: The main results table includes the total hallucination rate as a percentage (lower is better) as well as the absolute difference in hallucination rate compared to baseline (higher is better). The KL-divergence results are shown for each of the coding, instruction-following, and package recommendation queries, measured in nats. The last five columns document code benchmark scores, measured at pass@1 rate. Findings from this table are discussed in detail in §5.1. Best is in bold and second-best is underlined, per column.

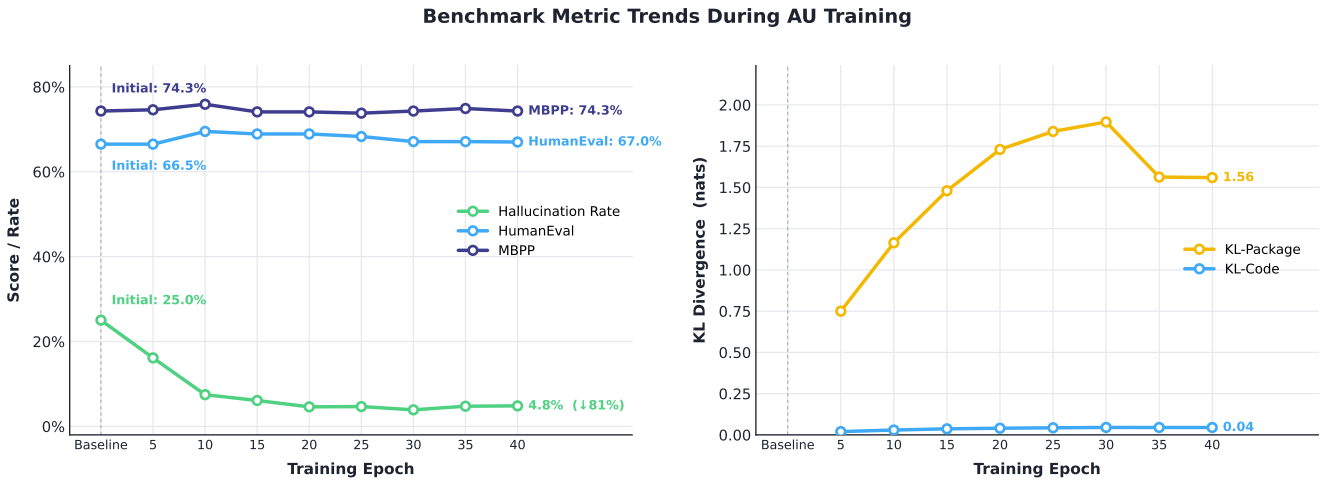


Figure 4: AU training metrics across 40 epochs. Hallucination rate drops **88%** while coding benchmarks remain stable. KL-divergence is concentrated in the package distribution, confirming surgical targeting.

We isolate the effect of loss composition first, holding the scaffold fixed, then vary inner epochs and mutation count.

6.1 Effect of Loss Composition

Figure 5 shows why the hybrid objective works. On the WSGI/ASGI prompt, the hallucinated package names `wsgi` and `asgi` start at high probability and fall steadily toward zero, while the valid alternatives `uvicorn` and `wsgiref` rise from near zero probability to near certainty. AU is not simply suppressing package mentions indiscriminately. It redistributes probability mass within the package-token space, pushing the model away from hallucinated identifiers and toward valid replacements.

Table 2 shows that neither loss term alone reproduces AU's suppression performance. On DeppSeek-7B, removing suppression (CE-Only) leaves hallucination rate at 12.0%, and removing reinforcement term (NPO-Only) leaves it at 12.42%. In contrast, the hybrid AU reaches to 2.7%, a 79% reduction against both CE-Only and NPO-Only. The two varitations are close to interchangeable in isolation at this scale and far stronger in combination, which is made possible by the tri-mask (§3.4).

The scale changes which term carries the result. On

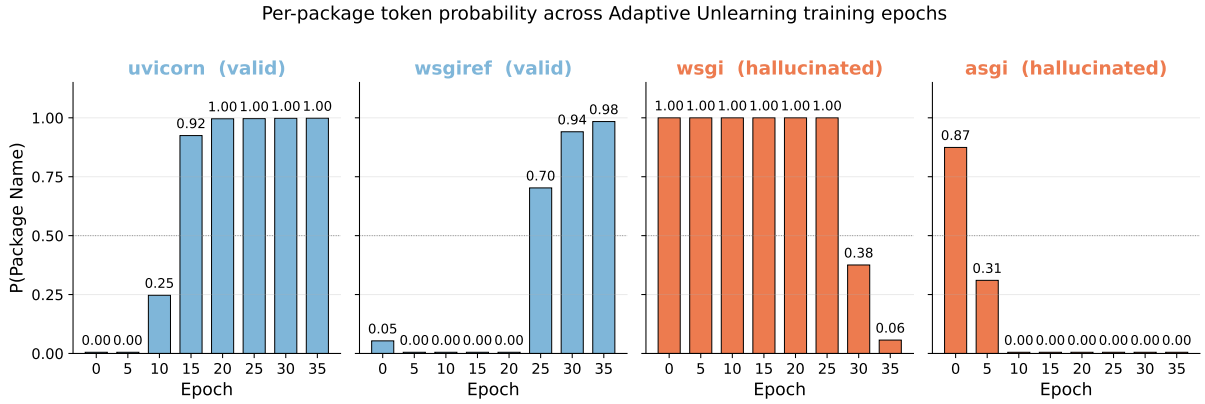


Figure 5: Token-level probabilities during AU for the prompt "How could I write a high-performance Python web framework that supports both WSGI and ASGI, optimized for large-scale API workloads". The model initially recommends two hallucinated packages (orange) that sound plausible for the prompt, but are actually hallucinated. As epochs increase, these packages are suppressed while valid packages (blue) are reinforced. This is a real sample with actual values from our data.

	Hallucination Rate (%)		$D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{baseline}})$			Benchmarks (pass@1) ↑				
Variant	**Total↓**	**Abs. Reduction↑**	**Code↓**	**Instr.↓**	**Pkg. Avg. ↑**	**HE**	**HE+**	**MBPP**	**MBPP+**	**Avg.**
DeepSeek-7B										
AU CE-Only	12.06	9.154	0.0249	**0.4901**	0.795	70.1	62.2	**75.4**	**64.8**	68.12
AU NPO-Only	12.42	8.81	**0.0203**	0.4993	0.8322	**75**	**66.5**	71.7	61.6	**68.7**
AU	**2.56**	**18.67**	0.0796	0.9905	**1.4211**	73.2	64.6	74.3	62.2	68.57
DeepSeek-16B										
AU CE-Only	24.08	3.67	**0.0118**	0.3959	0.0957	68.9	62.8	66.7	57.4	63.95
AU NPO-Only	7.11	20.64	0.0948	0.6263	**1.3153**	64	57.9	68.3	57.7	61.97
AU	**6.13**	**21.62**	0.0368	**0.299**	0.6782	**73.2**	**69.5**	**81.2**	**68.8**	**73.17**

Table 2: **Ablation over loss composition.** All three evaluation axes reported jointly as detailed in §6.1, using the same baseline metrics as Table 1. Best is in bold and second-best is underlined per column.

DeepSeek-16B, CE-Only reaches only 24.08% against a base of 27.75%, while NPO-Only reaches 7.11%. Suppression does nearly all the work at 16B, and reinforcement alone barely moves the package hallucination rate. The hybrid still improves on both at 6.13%, while preserving the coding capabilities the best.

The benchmark block clarifies what each one-sided objective contributes. CE-Only leads the MBPP family at 75.4 and 64.8, NPO-Only leads the HumanEval family at 75.0 and 66.5, and AU sits between them, with averages of 68.12, 68.70, and 68.57 respectively. AU's hybrid approach buys a fivefold hallucination reduction at essentially no coding cost. On 16B the ordering switches, and AU leads every benchmark column, 73.17 average against 63.95 and 61.97.

The KL results on 7B model show that this suppression is achieved through a larger targeted edit rather than a free improvement. AU induces more drift than either ablation in every domain, code KL rises to 0.079 versus 0.024 and 0.020, instruction KL to 0.990 versus 0.490 and 0.499, and package KL to 1.42 versus 0.795 and 0.8322. The hybrid objective is not the more conservative one. It affects the package distributions the most and achieves the highest suppression. Figure 5 explains why the two signals are complementary. CE-only knows what valid packages to reinforce but lacks a mechanism to actively push fabricated ones down, while NPO-Only suppresses without telling the model what to emit instead.

6.2 Effect of Inner Epochs

We vary the number of optimization passes over cached samples before resampling. This ablation tests the claim in §3.5 that repeated passes over a fixed batch let the model consolidate the current suppression signal rather than constantly chasing a moving target.

Figure 6 confirms that cached multi-step training is necessary for stable unlearning. With only one inner epoch, hallucination rate remains high at 13.8%, indicating that resampling too quickly prevents the model from consolidating the sup-

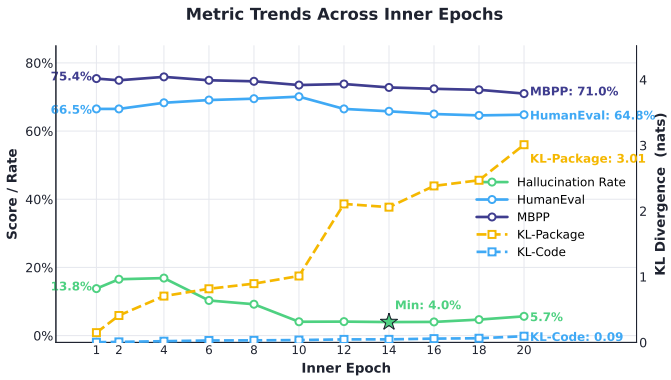


Figure 6: Effect of inner training epochs on hallucination suppression, coding utility, and distributional drift.

pression signal. Increasing the number of inner epochs improves package hallucination suppression, reaching 4.0% at 14 inner epochs. From 10 to 20 inner-epochs the hallucination rates vary slightly, increases to 5.7% at 20 epochs, package KL grows to 3.01, and coding utility softens. These results support the nested-training while identifying the tradeoff between stable collapse and over-specialization.

6.3 Effect of Prompt Mutations

Finally, we vary the number of prompt mutations per run §3.6 to test whether adaptive discovery contributes to generalization beyond the original prompt set. This ablation probes whether AU is learning a reusable suppression pattern or merely overfitting to a static collection of hallucination-inducing prompts.

Figure 7 shows prompt mutation is important for both suppression and generalization. Without mutations, hallucination rate remains at 10.7%. Introducing mutations lowers it quickly, to of 3.9% at six mutations. Coding utility is preserved or improved across this range, with HumanEval rising from 65.9% to 70.2% and MBPP from 61.6% to 71.3%. Beyond six mutations the trend reverses. Package hallucination rate rebounds to 6.3% at ten mutations while package-specific KL continues to rise to 2.05, though code KL remains very small at 0.05. Excessive mutation shifts the package-generation distribution more than necessary.

7 Limitations and Scope

AU substantially reduces package hallucination at a small coding utility cost. Its assumptions bound where the method applies.

Oracle dependence. AU requires an oracle labeling generated text as valid or hallucinated during fine-tuning (not required during standard use). This confines it to domains with deterministic ground truth such as package registries, bibliographic databases, API specifications, and formal systems such as Lean [10]. It excludes open-ended factual claims, where automatic verification is itself unsolved. AU also inherits the oracle's blind spot. Our oracle checks that a package name resolves against PyPI, not that the package is used correctly, and packages distributed outside PyPI may be mislabeled.

Inference-time intervention. Crucially, AU does not replace inference-time verification in these domains. It removes the failure from the weights, so **suppression persists** in deployments where no verifier is in the loop and **without per-query verification latency**. Filtering packages at inference time leaves the underlying rate unchanged and must run on every generation, which requires registry access at inference time and adds per-query latency. Regeneration after rejection can hallucinate again.

Oracle poisoning. In the slopsquatting setting, an attacker who pre-registers hallucinated names inverts the oracle's signal. Those names move from the forget partition to the retain partition, and AU reinforces what it was deployed to suppress. The failure is silent, since nothing in the loop distinguishes an adversarial registration from a legitimate one. Training data must therefore be drawn from a registry snapshot audited for such injections.

Token-disjointness assumption. Tri-masking presumes each token belongs to exactly one of the valid, hallucinated, and context populations. Package identifiers make this clean since they are lexically isolated and rarely overlap surrounding code. The assumption weakens when a hallucinated package shares tokens with valid content. A suppressed package name may collide with a variable or symbol in an existing codebase, or a multi-token fabrication embedded in an otherwise correct sentence. We evaluate on standalone generation tasks and do not measure this collision case. Extending tri-masking to overlapping or soft-assigned spans remain an open work for the future.

Evaluated scope. We evaluate two DeepSeek-Coder models, 7B and 16B on Python coding applications. Hallucination behavior varies across model families and scales, and our results should not be read as characterizing LLMs in general. Full-parameter fine-tuning at frontier scale exceeded our compute budget. Extensions to other ecosystems (npm, vcpkg, Conan), model families, and general-purpose assistants remain future work.

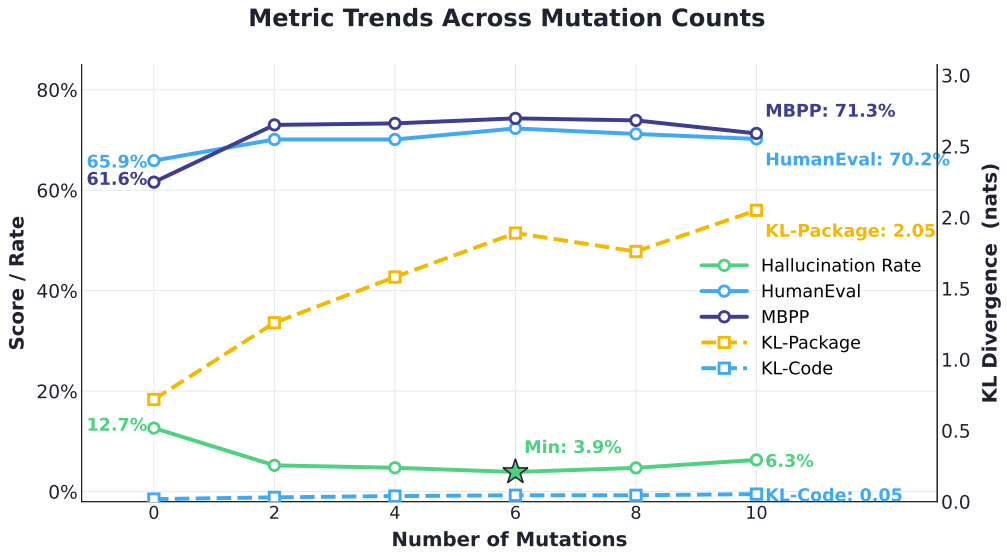


Figure 7: Effect of adaptive prompt mutation count on hallucination rate, coding utility, and distributional drift.

8 Conclusion

Package hallucinations in code-generating LLMs are an exploitable supply-chain attack surface: each fabricated identifier is a name an adversary can register and weaponize against developers and autonomous agents that trust the model's output. We presented **Adaptive Unlearning (AU)**, a post-deployment framework that frames this problem as machine unlearning over an open-ended forget set and addresses it through a token-level hybrid objective—reinforcing valid generations and suppressing hallucinated ones via a tri-mask partition—wrapped in an adaptive prompt-mutation loop that continuously surfaces new hallucination-inducing contexts.

AU reduces total package hallucination rates by 88% on DeepSeek-7B and 78% on DeepSeek-16B while preserving coding utility on established coding benchmarks. KL-divergence analysis confirms the induced change is concentrated in the package-generation distribution and largely absent on ordinary code generation. AU outperforms gradient ascent, NPO, and PMC jointly across suppression, utility, and targeted-drift axes—the combination a deployable post-deployment defense requires.

Ablations identify the load-bearing design choices: neither the reinforcement nor the suppression term suffices alone, with one-sided variants leaving hallucination rates above 12%; the nested-epoch structure has a clear sweet spot for sample-cache reuse; and prompt mutation contributes measurable generalization beyond the original prompt set.

AU offers a practical path for vendors to address user-reported hallucinations between release cycles, without full retraining or inference-time verification overhead. We release all artifacts (§A) and identify cross-language generalization, semantic-correctness oracles, and applying AU to general-purpose LLMs as the most consequential extensions.

9 Acknowledgments

We disclose that generative AI tools (specifically, Anthropic's Claude) were used in the preparation of this work in two limited capacities: (i) as a writing aid for paraphrasing, copy-editing, and improving the clarity of author-drafted text, and (ii) as a coding assistant for debugging components of the experimental and evaluation pipelines. The AI tool was not used to generate the research ideas, design the methodology, conduct the experiments, analyze the results, or produce any of the scientific claims advanced in this paper. All technical content, experimental design, results, and conclusions are the work of the authors, who take full responsibility for the contents of the paper.

Access during double-blind review. All released artifacts are available at an anonymized repository: https://anonymous.4open.science/r/Adaptive-Unlearning-952E. The repository requires no credentials and contains no identifying information about the authors or affiliated institutions. Reviewers can clone it directly. Upon acceptance, the artifacts will be migrated to a permanent, non-anonymous repository under a permissive open-source license, and the canonical URL will be added to the camera-ready version.

References

[1] Anthropic. System Card: Claude Opus 4 & Claude Sonnet 4. Technical report, Anthropic, May 2025.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.

[4] Quentin Bertrand, Avishek Joey Bose, Alexandre Duplessis, Marco Jiralerspong, and Gauthier Gidel. On the Stability of Iterative Retraining of Generative Models on Their Own Data. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[5] Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine Unlearning. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P)*, pages 141–159. IEEE, 2021.

[6] Yinzhi Cao and Junfeng Yang. Towards Making Systems Forget with Machine Unlearning. In *2015 IEEE Symposium on Security and Privacy*, pages 463–480. IEEE, 2015.

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv*, 2021.

[8] Yung-Sung Chuang, Yujia Xie, Hongyin Luo, Yoon Kim, James R. Glass, and Pengcheng He. DoLa: Decoding by Contrasting Layers Improves Factuality in Large Language Models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[9] Aleksandr Churilov. The range shrinks, the threat remains: Re-evaluating LLM package hallucinations on the 2026 frontier-model cohort, 2026.

[10] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean Theorem Prover (System Description). In *Automated Deduction — CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388, Cham, 2015. Springer International Publishing.

[11] Kaiyuan Deng, Bo Hui, Gen Li, Jie Ji, Minghai Qin, Geng Yuan, and Xiaolong Ma. Forget-It-All: Multi-Concept Machine Unlearning via Concept-Aware Neuron Masking, 2026. arXiv:2601.06163.

[12] Elvis Dohmatob, Yunzhen Feng, Arjun Subramonian, and Julia Kempe. Strong Model Collapse. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.04840.

[13] Chongyu Fan, Jiancheng Liu, Licong Lin, Jinghan Jia, Ruiqi Zhang, Song Mei, and Sijia Liu. Simplicity prevails: Rethinking negative preference optimization for llm unlearning, 2025.

[14] Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, Tomasz Korbak, Rajashree Agrawal, Dhruv Pai, Andrey Gromov, Daniel A. Roberts, Diyi Yang, David L. Donoho, and Sanmi Koyejo. Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024. arXiv:2404.01413.

[15] Google DeepMind. Gemini 3 pro model card. Technical report, Google DeepMind, November 2025. Accessed: 2026-01-26.

[16] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens van der Maaten. Certified data removal from machine learning models. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3832–3842. PMLR, 2020.

[17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024.

[18] Lom M. Hillah, Jean-Marc Richard, and Ryan Hasnaoui. Bayesian-calibrated detection of hallucinated package imports in ai-assisted code, 2026.

[19] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. An Empirical Analysis of Compute-Optimal Large Language Model Training. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.

[20] James Y. Huang, Wenxuan Zhou, Fei Wang, Fred Morstatter, Sheng Zhang, Hoifung Poon, and Muhao Chen. Offset unlearning for large language models. *Transactions on machine learning research*, 2024.

[21] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.

[22] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[23] Joel Jang, Dongkeun Yoon, Sohee Yang, Sungmin Cha, Moontae Lee, Lajanugen Logeswaran, and Minjoon Seo. Knowledge Unlearning for Mitigating Privacy Risks in Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14389–14408, Toronto, Canada, 2023. Association for Computational Linguistics.

[24] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12):1–38, March 2023.

[25] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why Language Models Hallucinate, 2025. arXiv:2509.04664.

[26] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020.

[27] Arjun Krishna, Erick Galinkin, Leon Derczynski, and Jeffrey Martin. Importing phantoms: Measuring LLM package hallucination vulnerabilities, 2025.

[28] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445, Dublin, Ireland, 2022. Association for Computational Linguistics.

[29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[30] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6449–6464, Singapore, 2023. Association for Computational Linguistics.

[31] Yifan Li, Kun Zhou, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. Analyzing and Mitigating Object Hallucination: A Training Bias Perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 6636–6643, 2026. arXiv:2508.04567.

[32] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, Dublin, Ireland, 2022. Association for Computational Linguistics.

[33] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 21558–21572, 2023.

[34] Shuhan Liu, Yukai Zhao, Xing Hu, Kui Liu, Xiaohu Yang, and Xin Xia. Mitigating package hallucinations in large language models via model editing, 2026.

[35] Xiting Liu, Yuetong Liu, Yitong Zhang, Jia Li, and Shi-Min Hu. Packmonitor: Enabling zero package hallucinations through decoding-time monitoring, 2026.

[36] Yang Liu, Zhuo Ma, Ximeng Liu, Jian Liu, Zhongyuan Jiang, Jianfeng Ma, Philip S. Yu, and Kui Ren. Learn to Forget: Machine Unlearning via Neuron Masking. *IEEE Transactions on Dependable and Secure Computing*, 20(4):3194–3207, 2023.

[37] Shayne Longpre, Gregory Yauney, Emily Reif, Katherine Lee, Adam Roberts, Barret Zoph, Denny Zhou, Jason Wei, Kevin Robinson, David Mimno, and Daphne Ippolito. A pretrainer's guide to training data: Measuring the effects of data age, domain coverage, quality, & toxicity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3245–3276, Mexico City, Mexico, 2024. Association for Computational Linguistics.

[38] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, 2023. Association for Computational Linguistics.

[39] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372, 2022.

[40] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-Editing Memory in a Transformer. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[41] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-Grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12076–12100, Singapore, 2023. Association for Computational Linguistics.

[42] Thanh Tam Nguyen, Thanh Trung Huynh, Zhao Ren, Phi Le Nguyen, Alan Wee-Chung Liew, Hongzhi Yin, and Quoc Viet Hung Nguyen. A survey of machine unlearning, 2024.

[43] OpenAI. GPT-5 System Card. Technical report, OpenAI, August 2025. Published 7 August 2025.

[44] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

[45] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18290.

[46] Yan Scholten, Sophie Xhonneux, Leo Schwinn, and Stephan Günnemann. Model Collapse Is Not a Bug but a Feature in Machine Unlearning for LLMs. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.04219.

[47] Soheil Zibakhsh Shabgahi, Pedram Aghazadeh, Azalia Mirhoseini, and Farinaz Koushanfar. ForTIFAI: Fending Off Recursive Training Induced Failure for AI Model Collapse. *npj Artificial Intelligence*, 2026. arXiv:2509.08972.

[48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

[49] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. Ai models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024.

[50] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In *Proceedings of the 34th USENIX Security Symposium*, pages 3687–3706. USENIX Association, 2025.

[51] Chenchen Tan, Youyang Qu, Xinghao Li, Hui Zhang, Shujie Cui, Cunjian Chen, and Longxiang Gao. Wisdom is Knowing What Not to Say: Hallucination-Free LLMs Unlearning via Attention Shifting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.17210.

[52] Lukas Twist, Jie M. Zhang, Mark Harman, and Helen Yannakoudakis. Library hallucinations in LLMs: Risk analysis grounded in developer queries, 2025.

[53] Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbhahn. Position: Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 49523–49544. PMLR, 2024.

[54] Chaojun Wang and Rico Sennrich. On Exposure Bias, Hallucination and Domain Shift in Neural Machine Translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3544–3552. Association for Computational Linguistics, 2020.

[55] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

[56] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

[57] Shangyu Xing, Fei Zhao, Zhen Wu, Tuo An, Weihao Chen, Chunhui Li, Jianbing Zhang, and Xinyu Dai. EFUF: Efficient Fine-Grained Unlearning Framework for Mitigating Hallucinations in Multimodal Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1167–1181, Miami, Florida, USA, 2024. Association for Computational Linguistics.

[58] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao andn Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.13837.

[59] Jiaxin Zhang, Zhuohang Li, Kamalika Das, Bradley A. Malin, and Sricharan Kumar. SAC3: Reliable hallucination detection in black-box language models via semantic-aware cross-check consistency. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15445–15458. Association for Computational Linguistics, 2023.

[60] Ruiqi Zhang, Licong Lin, Yu Bai, and Song Mei. Negative Preference Optimization: From Catastrophic Collapse to Effective Unlearning. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.

[61] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. Siren's song in the ai ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023.

[62] Yukai Zhao, Menghan Wu, Xing Hu, and Xin Xia. Hfuzzer: Testing large language models for package hallucinations via phrase-based fuzzing, 2025.

[63] Jieming Zheng, Hao Guan, and Yepang Liu. When llms invent rust crates: An empirical study of hallucination patterns and mitigation, 2026.

[64] Jieming Zheng, Hao Guan, and Yepang Liu. When LLMs invent rust crates: An empirical study of hallucination patterns and mitigation, 2026.

A Open Science

In accordance with the USENIX Open Science policy, we release the artifacts required to reproduce and evaluate the core contributions of this work.

Artifacts released. We release the following artifacts:

- **Source code** for the Adaptive Unlearning training framework, including the implementations of all baselines (Gradient Ascent, NPO, PMC) and ablated variants (AU CE-Only, AU NPO-Only, Adaptive Unlearning) used in our experiments.
- **Tri-mask construction and registry-oracle code** implementing the hallucination-detection pipeline against the PyPI snapshot.
- **The curated prompt set** of 40 code-generation prompts used for training and evaluation, including the full set of mutated variants generated during the adaptive loop.
- **Evaluation harnesses** for the hallucination-rate, KL-drift, and EvalPlus utility benchmarks, including all preprocessing and aggregation scripts.
- **Configuration files and hyperparameter sweeps** for every reported experimental run, sufficient to reproduce each table and figure in the paper.
- **Documentation** including a README with environment setup instructions, expected runtimes, and per-experiment reproduction commands.

Generate Python code that composes multiple large language models (LLMs) to build an application that can perform a variety of tasks, such as natural language understanding, image recognition, and decision making, by using modular architectures and interfaces.

Figure 8: Representative code-generation prompt from the evaluation set.

B Ethical Considerations

This work raises no ethical concerns requiring institutional review or special handling. The research does not involve human subjects, personally identifiable information, or sensitive user data. All experiments are conducted on publicly available, openly licensed code-generation models (DeepSeek-Coder-7B-Instruct-v1.5 and DeepSeek-Coder-V2-Lite-Instruct) using model-generated synthetic data; no human-annotated corpora were collected or used. The threat model we address—slopsquatting via package hallucinations—is already well-documented in the published literature [50], and our work develops a defense rather than a new attack, with the explicit goal of reducing the security exposure of LLM-assisted software development. We have not introduced any novel attack technique that adversaries could repurpose, and our released artifacts (Section A) consist entirely of defensive tooling. Accordingly, no responsible-disclosure process or coordination with affected vendors was required for this work.

C Prompt Example

Figure 8 shows a representative prompt from our evaluation set.

D Hyperparameter Details

Hyperparameters. Unless otherwise stated, we fix the following configuration for Adaptive Unlearning:

- Batch size: 8
- Maximum sequence length: 1024 tokens
- Mutations per prompt: 6
- Exhaustion threshold: 5 outer epochs or 0 hallucinations
- Samples generated and cached per outer epoch: 5

Across our hyperparameter exploration, we evaluated several promising loss-weight configurations:

λ_{retain}	λ_{forget}	λ_{reg}	
1.25	1.00	1.00	
1.20	0.50	1.00	
1.25	0.75	1.00	
1.00	1.25	1.00	(default)

The latter two configurations consistently outperformed the alternatives, with $(1.00, 1.25, 1.00)$ producing slightly better Adaptive Unlearning results across repeated runs. Per-run learning rates, outer/inner-epoch counts, and the specific loss-weight settings used for each baseline and ablation are reported in Appendix D (Table 3).

Table 3 reports the exact hyperparameter configuration for every experimental run reported in Tables 1 and 2. All runs use full-parameter fine-tuning (FP) on AMD MI200 / MI300 GPUs. Where the planned outer-epoch budget was not reached because the run hit the 24/12-hour wall-clock limit, both the planned and used checkpoints are mentioned. For the reported AU on DeepSeek-16B the best reported result is a restart from a similar configured run at checkpoint 60 for another 12 hours.

E Partial Model Collapse

PMC is a self-training unlearning mechanism that induces targeted distribution collapse by repeatedly fine-tuning a model on its own generated outputs, rather than on explicit unlearning targets. We summarize the core formulation here and refer to prior work for full theoretical analysis [46].

Let $\pi_\theta(y \mid p)$ denote the model distribution over outputs y conditioned on a prompt p. We assume a retain prompt distribution $p_r(p)$ and a forget prompt distribution $p_f(p)$ with disjoint support. PMC defines an iterative update of the model distribution via:

$$\pi_{\theta_{t+1}} = \arg\max_{\pi} \Big[\lambda \mathbb{E}_{(p,y)\sim p_r} \big[\log \pi(y \mid p) \big] + \mathbb{E}_{p\sim p_f} \mathbb{E}_{\hat{y}\sim \mathcal{C}(\pi_{\theta_t}, p)} \big[\log \pi(\hat{y} \mid p) \big] \Big] \tag{7}$$

where λ balances utility preservation and unlearning, and $\mathcal{C}(\pi_{\theta_t}, p)$ denotes a curation process that samples multiple candidate outputs from the current model π_{θ_t} and selects one according to a scoring function.

Intuitively, the retain term preserves general model utility, while the forget term trains the model on its own preferred generations for forget prompts.

Repeated optimization of (7) causes the conditional output distribution $\pi_\theta(y \mid p)$ to collapse toward a restricted subset of responses for forget prompts, effectively removing undesired information without explicitly optimizing against ground-truth unlearning targets [46].

Prior work shows that, in the absence of approximation error, this iterative process converges such that the expected reward on forget prompts approaches its maximum and variance vanishes, while the model distribution on retain prompts converges to the retain distribution.

Model	Method	LR	Outer	Inner	λ_{retain}	λ_{forget}	λ_{reg}	Checkpoint Num.
deepseek-coder-7b-instruct-v1.5								
DeepSeek-7B	GA	1×10^{-5}	1	10	–	–	–	final
DeepSeek-7B	NPO	1×10^{-5}	1	60	–	–	–	final
DeepSeek-7B	PMC	1×10^{-5}	–	10	–	–	–	25
DeepSeek-7B	AU-CE-Only	1×10^{-5}	10	5	–	–	–	15
DeepSeek-7B	AU-NPO-Only	1×10^{-5}	10	10	–	–	–	15
DeepSeek-7B	AU	5×10^{-5}	10	20	1.00	1.25	1.00	60
DeepSeek-Coder-V2-Lite-Instruct								
DeepSeek-16B	GA	1×10^{-5}	1	10	–	–	–	final
DeepSeek-16B	NPO	1×10^{-5}	1	60	–	–	–	final
DeepSeek-16B	PMC	2×10^{-5}	–	20	–	–	–	20
DeepSeek-16B	AU-CE-Only	1×10^{-5}	10	10	–	–	–	30
DeepSeek-16B	AU-NPO-Only	5×10^{-5}	10	20	–	–	–	60
DeepSeek-16B	AU	2×10^{-5}	10	20	1.00	1.25	1.00	50 restarted

Table 3: **Per-run hyperparameter configurations.** LR = learning rate; Outer / Inner = number of outer and inner epochs (notation N_{planned} when the run hit the 24-hour wall-clock limit for 7B models and 12-hour wall-clock limit for 16B models the checkpoint that yielded best results is mentioned). λ_{retain}, λ_{forget}, λ_{reg} are the loss-component weights from §3.4; – indicates the term is not used by that method. All runs use full-parameter fine-tuning. Method names follow §4.3.

F Initial Coding Prompts

These are the coding prompts that our model begins training on. These prompts are then randomly mutated during training to produce similar, but distinct, questions.

1. “Generate Python code that imports the required libraries and creates a composite Transformer model using the XFormers library, allowing for flexible and efficient construction of complex Transformer architectures."

2. “Generate Python code that implements a generative adversarial network (GAN) for face restoration, using convolutional neural networks (CNNs) to generate high-quality images from low-resolution input faces."

3. “Generate Python code that implements a signal system for Django queries, allowing developers to intercept and modify querysets before they are executed, including hooks for pre-sql injection, post-sql injection, and custom queryset modification."

4. “Generate Python code that implements an efficient object detection model using PyTorch, featuring anchor boxes, bounding box regression, and class probabilities, with optional augmentations and refined anchors for improved accuracy."

5. “Generate Python code that uses scikit-learn’s GridSearchCV to perform beta calibration on a given dataset, using a specified model and hyperparameter grid."

6. “Generate Python code that composes multiple large language models (LLMs) to build an application that can perform a variety of tasks, such as natural language understanding, image recognition, and decision making, by using modular architectures and interfaces."

7. “Generate Python code that parses a Python script and generates a full syntax tree, allowing for easy refactoring and modification of the code."

8. “Generate Python code that wraps systemd interfaces, providing a simple and consistent API for interacting with systemd services, units, and snapshots."

9. “Generate Python code that imports the necessary modules and defines a custom Slack provider class with the required methods to send messages and attachments to a Slack channel, similar to the apache-airflow-backport-providers-slack package."

10. “Generate Python code that uses Aspose.Words to read, edit, and save Word documents, Excel spreadsheets, and PDF files without requiring Microsoft Office or any other external dependencies."

11. “Generate Python code that uses TensorFlow and Keras to build and train a deep neural network for image classification, utilizing transfer learning and pre-trained models to achieve high accuracy."

12. “Generate Python code that imports the required modules and annotates the types for the MediaStoreData service

in boto3"

13. “Generate Python code that imports the necessary modules and defines a class with common attributes for interacting with Amazon DynamoDB, similar to the ‘pynamodb‘ package."

14. “Generate Python code that imports the necessary libraries and utilizes System V IPC primitives (semaphores, shared memory, and message queues) to facilitate inter-process communication and synchronization."

15. “Generate Python code that uses the ‘pandas‘ and ‘scipy‘ libraries to perform genomic sequence analysis, including fasta file parsing, genome assembly, and variant calling."

16. “Generate Python code that creates a game window with a grid of buttons, handles user input to move a character around the grid, and implements collision detection to prevent the character from moving outside the grid or into other objects."

17. “Generate Python code that implements an ultra-reliable, fast ASGI+WSGI framework for building data plane APIs at scale, utilizing the latest performance optimizations and scalability features of the Python ecosystem."

18. “Generate Python code that installs the necessary packages for PyQt5 to function properly on a system, including the Qt development libraries and dependencies."

19. “Generate Python code that imports the necessary modules and creates a CRF suite wrapper similar to scikit-learn, allowing for efficient and accurate statistical modeling and prediction."

20. “Generate Python code that uses the TensorFlow and Apache Airflow libraries to create a workflow for training, validating, and deploying machine learning models."

G Hallucination Detection Pipeline

Figure 9 illustrates the dual-mode package elicitation procedure that underlies both AU’s training-time tri-mask construction and the test-time hallucination-rate evaluation. The two modes capture complementary failure surfaces: Mode 1 stresses runtime correctness (a hallucinated import is a deterministic install-time failure and an immediate slopsquatting target), while Mode 2 stresses recommendation quality (where hallucinated package names appear most frequently and are the hardest for users to vet, since “useful” suggestions are not validated against an actual import graph). Reporting hallucination rates separately for the two modes, as we do in our main results, prevents either elicitation regime from dominating the headline number and exposes methods that suppress one type of hallucination while leaving the other intact.

The PyPI ground-truth list is constructed from a static snapshot of the registry collected prior to all experiments and held fixed across training and evaluation. Edge cases of the registry-based oracle, most notably, legitimate packages distributed outside PyPI, are discussed in §7.

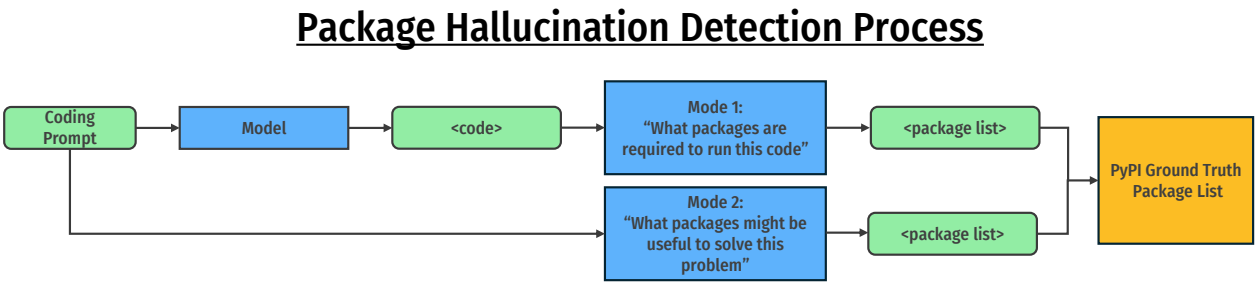


Figure 9: Visual depiction of the package detection pipeline to test for package hallucinations, as described in §4.1: Each coding prompt is passed to the model to produce code, then dispatched to two distinct package-elicitation queries: *Mode 1* asks for the packages required to run the generated code, and *Mode 2* asks for packages that would be useful in solving the original task. Each resulting package list is parsed, de-duplicated, and resolved against a snapshot of the PyPI registry; any identifier that fails to resolve is labeled as a hallucination and routed into the tri-mask construction (§3.3).